
Comunicação Otimizada para Redes Industriais em Edge Computing

Um ecossistema de comunicação segura com compressão zlib e criptografia
TLS 1.3 transparentes.

Ruan Felipe Lauriano Ramos

Departamento de Ciência da Computação · Boa Vista — RR

Prof. Herbert Oliveira Rocha

github.com/ruanflau

Roteiro da apresentação

Projeto final de Sistemas Operacionais I — Tema 1. A apresentação segue a mesma estrutura do artigo entregue.

01 Contexto e problema

02 Arquitetura e desenvolvimento

03 Implementação dos conceitos de SO

04 Estudo de casos

05 Análise de performance

06 Demonstração ao vivo

07 Conclusão

A borda industrial opera sob canais instáveis e caros

Subestações, plataformas petrolíferas e plantas de saneamento coletam telemetria contínua e exigem diagnóstico remoto — por links de satélite ou rádio de banda limitada. Três problemas precisavam ser resolvidos.

CONFIDENCIALIDADE

Texto claro é interceptável

Dados sem proteção ficam vulneráveis a ataques Man-in-the-Middle no enlace.

EFICIÊNCIA

Banda limitada e cara

Transmitir sem compressão eleva desnecessariamente os custos operacionais.

CONTROLE

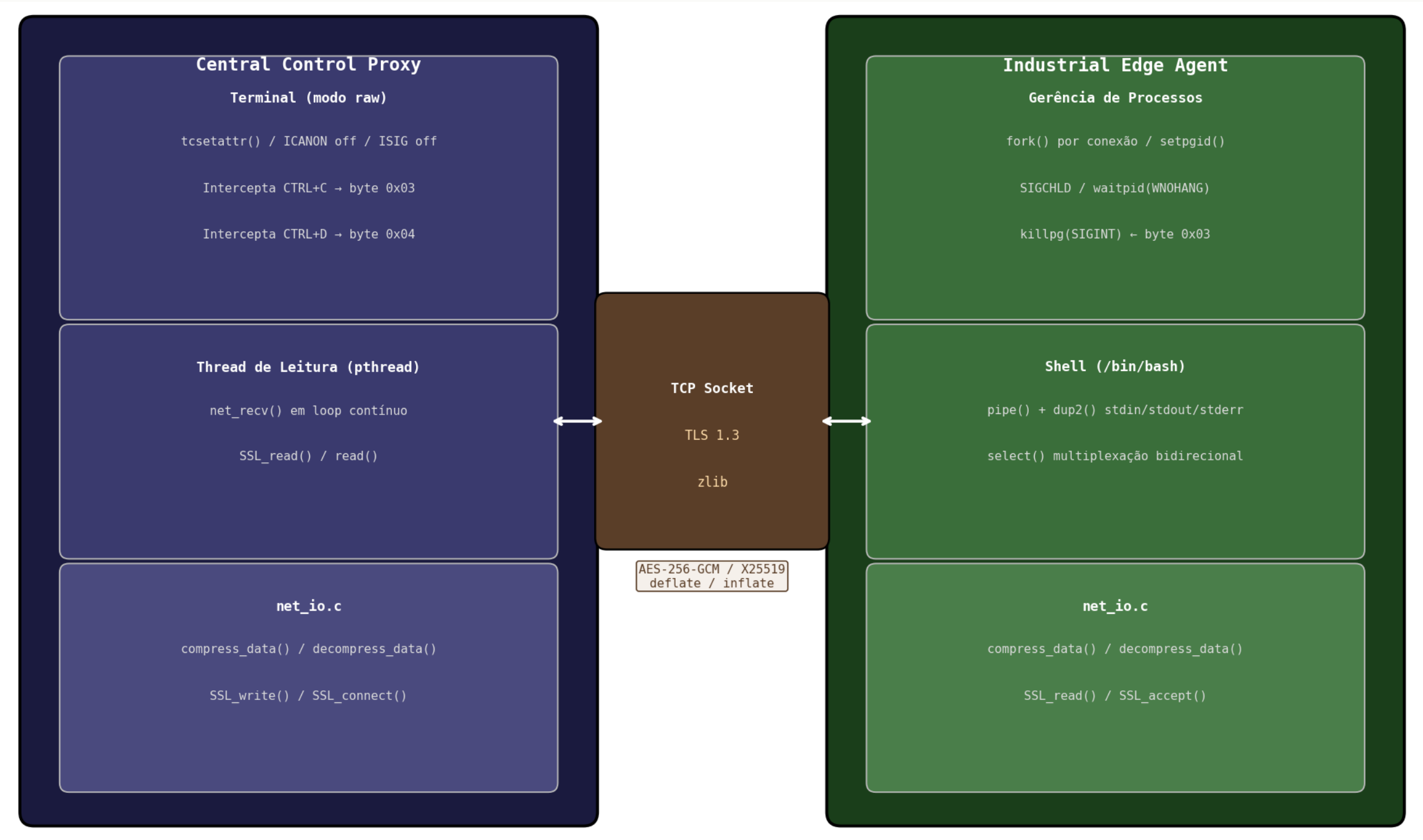
Sinais não cruzam a rede

CTRL+C e CTRL+D não funcionam pela rede por padrão.

Restrição do enunciado: resolver tudo isso **sem modificar o código-fonte do interpretador de comandos.**

Fluxo de dados unificado em net_io.c

Dois binários nativos em C — `industrial_agent` (servidor) e `control_proxy` (cliente) — compartilham a camada de I/O `net_io.c`.



Sete módulos em C, responsabilidades isoladas

common.h/c	Constantes globais, struct AppConfig e parser de CLI (--port · --host · --compress · --encrypt · --log)
compress.c/h	Pipeline de compressão e descompressão via zlib (deflate / inflate)
tls.c/h	Inicializa contextos SSL de servidor e cliente; abstrai a complexidade do OpenSSL
logger.c/h	Grava cada transação como SENT / RECEIVED N bytes, com escape hexadecimal
net_io.c/h	Camada unificada: decide SSL_write/write e aplica compress_data
agent.c · proxy.c	Binários principais: Industrial Edge Agent (servidor) e Central Control Proxy (cliente)

Industrial Edge Agent — quatro camadas concorrentes

01 Loop de aceitação

`socket` → `bind` → `listen` → `accept`; a cada conexão um `fork()` isola a sessão no filho.

02 Criação do shell

Segundo `fork()` cria o `bash`; `pipe()` + `dup2()` redirecionam `stdin/out/err`.

03 Multiplexação `select()`

Monitora o socket de rede e o pipe de saída do `bash` simultaneamente, sem bloquear em nenhum canal.

04 Tratamento de sinais

`0x03`→`killpg(SIGINT)`, `0x04`→`EOF`; `SIGCHLD` + `waitpid(WNOHANG)` evita zumbis.

Central Control Proxy — três responsabilidades

CONEXÃO E CRIPTOGRAFIA

TLS antes do tráfego

Conecta via TCP; com `--encrypt`, o `SSL_connect()` ocorre antes de qualquer byte. A partir daí, tudo passa pelo OpenSSL.

MODOS RAW DE TERMINAL

`tcsetattr()`

Desliga ICANON e ISIG — cada tecla é imediata e CTRL+C/D viram bytes puros. Estado restaurado via `atexit()`.

THREAD DE LEITURA SSL

pthread dedicada

O `select()` não enxerga o buffer interno do OpenSSL. Uma thread chama `SSL_read()` em loop e elimina o travamento.

Duas camadas opcionais, transparentes ao shell

COMPRESSÃO · `zlib`

Dois pipelines simétricos

`deflateInit` → `deflate` → `deflateEnd` na escrita e o inverso na leitura.

`net_io.c` aplica `compress_data` antes de cada envio — transparente para `agent.c` e `proxy.c`.

CRIPTOGRAFIA · `OpenSSL TLS 1.3`

Validada com `openssl s_client`:

Protocolo	TLS 1.3
Cifra	AES_256_GCM_SHA384
Troca de chaves	X25519 · forward secrecy
Chave pública	RSA 4096 bits

Sete testes funcionais — todos aprovados

Teste	Resultado
4.1 — Conexão básica (ls, pwd, uname -a)	✓ Aprovado
4.2 — Compressão --compress	✓ Aprovado
4.3 — Criptografia --encrypt (TLS 1.3)	✓ Aprovado
4.4 — Ambas as flags + log de auditoria	✓ Aprovado
4.5 — CTRL+C remoto interrompe sleep 60	✓ Aprovado
4.6 — Múltiplos clientes simultâneos isolados	✓ Aprovado

4.7 — VALGRIND · MEMÓRIA

0 bytes definitely lost

0 errors em ambos os binários. Ubuntu 24.04 · kernel 6.17.0.

– DEMONSTRAÇÃO AO VIVO

Quatro passos no terminal

PASSO 1 · CONEXÃO BÁSICA

```
$ ./industrial_agent --port=9095  
$ ./control_proxy --host=127.0.0.1 --port=9095  
  
Digitar ls · pwd · uname -a
```

PASSO 2 · SINAL CTRL+C

```
$ sleep 60  
# pressionar CTRL+C  
  
O sleep é interrompido remotamente.
```

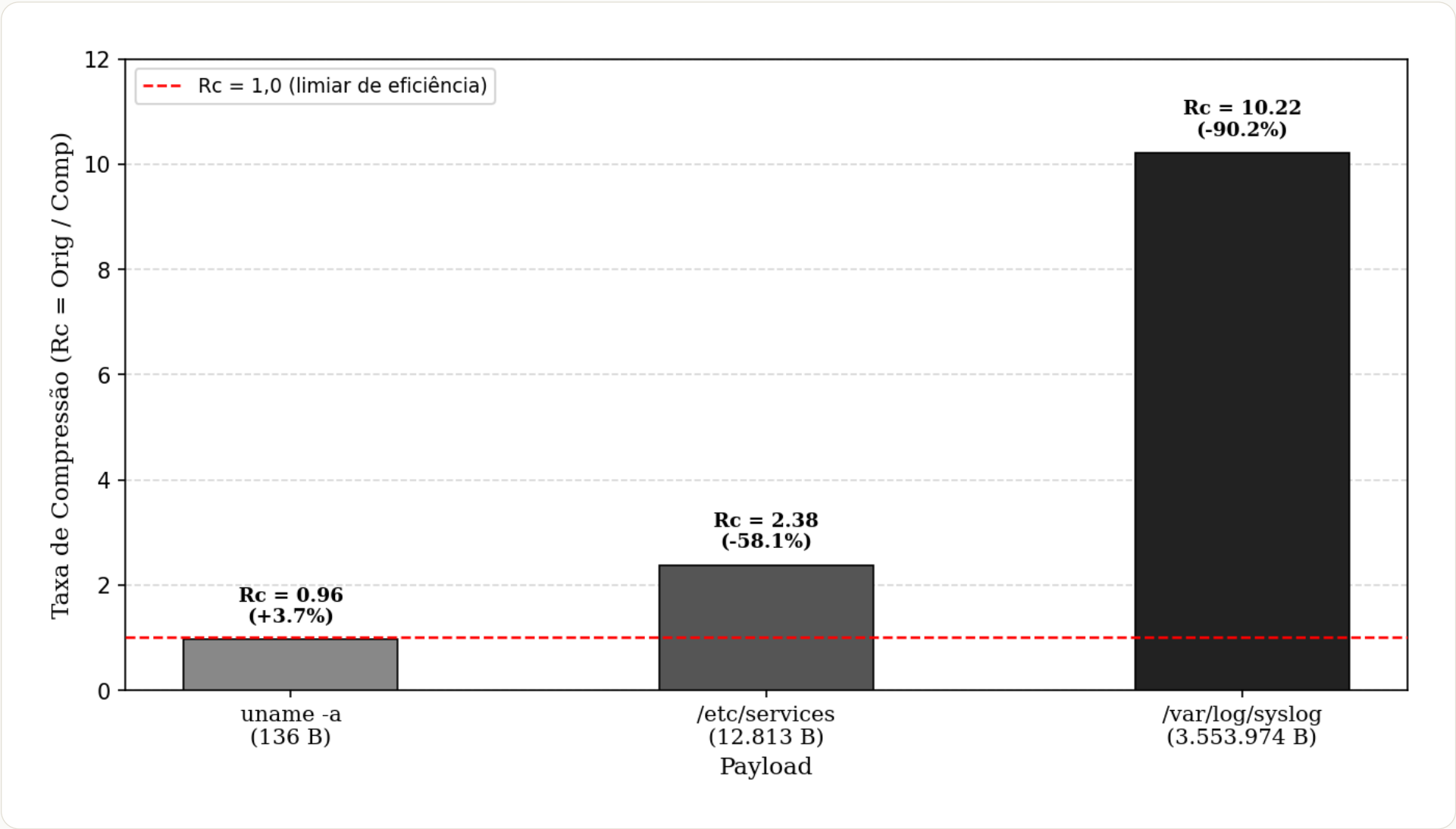
PASSO 3 · MODO COMPLETO

```
$ ./industrial_agent --port=9095 --compress --encrypt  
$ ./control_proxy ... --compress --encrypt --log  
  
Digitar top -b -n 1 · df -h
```

PASSO 4 · LOG GERADO

```
$ cat logs/auditoria.log  
  
Transações no formato SENT / RECEIVED.
```

Taxa de compressão por tipo de payload



uname -a • 136 B Rc 0,96 • +3,7%

Ficou maior: o cabeçalho fixo do deflate (~18 B) supera o ganho. Comprimir deve ser seletivo.

/var/log/syslog • 3,5 MB Rc 10,22 • -90,2%

Logs têm altíssima repetição estrutural — o deflate aproveita cada byte repetido.

CAPTURA REAL • tcpdump

8,9 KB → 4,6 KB • -48,3%

Criptografia praticamente gratuita

10 CONEXÕES SEQUENCIAIS

≈ 0 %

de overhead de CPU

Sem TLS	10,099 s · 1,010 s/conexão
Com TLS 1.3	10,052 s · 1,005 s/conexão

Handshake em 1 round-trip

TLS 1.3 precisa de apenas 1 round-trip contra 2 do TLS 1.2 — metade da latência de estabelecimento.

X25519 em milissegundos

Em loopback, o handshake consome poucos ms — desprezível frente ao tempo total de sessão em redes reais.

Na prática, ativar `--encrypt` não introduz custo perceptível de latência.

Do campo à produção

APLICAÇÃO PRÁTICA

Subestações via satélite

O `industrial_agent` roda no gateway; o engenheiro conecta com `--compress` e `--encrypt`.

Auditoria para conformidade

Nos testes: 13 transações e 26.840 bytes, incluindo a saída de `top` e `df -h`.

LIMITAÇÕES DOCUMENTADAS

Ausência de framing

Sem `length-prefixing`, o TCP pode fragmentar e quebrar a descompressão. Solução: prefixar 4 bytes de tamanho a cada bloco.

Certificado autoassinado

`SSL_VERIFY_NONE` só serve a testes. Em produção: `SSL_VERIFY_PEER` contra uma CA confiável.

Conceitos de SO, aplicados sem frameworks de alto nível

Todos os requisitos atendidos: shell remoto transparente sem modificar o bash, compressão dinâmica, criptografia de transporte, sinais POSIX pela rede, log de auditoria e código sem vazamentos — usando processos, pipes, sinais, sockets e sincronização.

10,22×

compressão em logs

48,3%

menos tráfego (tcpdump)

≈0%

overhead TLS 1.3

0

bytes lost (Valgrind)